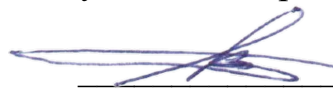


MSU ROVER TEAM

**ОТКРЫТЫЕ БИБЛИОТЕКИ ДЛЯ АВТОНОМНОЙ НАВИГАЦИИ
ШЕСТИКОЛЕСНОГО ПРОТОТИПА МАРСОХОДА (РОВЕРА) ПО УМЕРЕННО
ПЕРЕСЕЧЁННОЙ НЕЗНАКОМОЙ МЕСТНОСТИ С ВИЗУАЛЬНЫМ
РАСПОЗНАВАНИЕМ ЦЕЛИ НАВИГАЦИИ.**

СОГЛАСОВАНО:

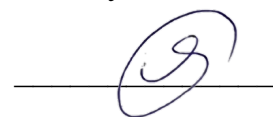
Научный эксперт проекта, к.ф.-м.н.

 В.М. Буданов

05.12.2025

УТВЕРЖДАЮ:

Руководитель проекта

 А.А. Смирнов

05.12.2025

**Открытая библиотека автономной навигации по распознанным указателям
движения.**

Руководство программиста.

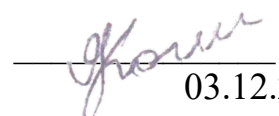
ЛИСТ УТВЕРЖДЕНИЯ

MSUROVERTEAM-SLAM-V1.0.0

(открытая библиотека в сети Интернет)

ИСПОЛНИТЕЛИ:

 В.К. Егоров
03.12.2025

 Я.А. Коломиец
03.12.2025

MSU ROVER TEAM

УТВЕРЖДЕНО

**ОТКРЫТЫЕ БИБЛИОТЕКИ ДЛЯ АВТОНОМНОЙ НАВИГАЦИИ
ШЕСТИКОЛЕСНОГО ПРОТОТИПА МАРСОХОДА (РОВЕРА) ПО УМЕРЕННО
ПЕРЕСЕЧЁННОЙ НЕЗНАКОМОЙ МЕСТНОСТИ С ВИЗУАЛЬНЫМ
РАСПОЗНАВАНИЕМ ЦЕЛИ НАВИГАЦИИ.**

**Открытая библиотека автономной навигации по распознанным указателям
движения.**

Руководство программиста.

MSUROVERTEAM-SLAM-V1.0.0

(открытая библиотека в сети Интернет)

Листов 24.

АННОТАЦИЯ.

Открытая библиотека автономной навигации по распознанным указателям движения разработана в рамках проекта «Разработки открытых библиотек для автономной навигации шестиколесного прототипа марсохода (ровера) по умеренно пересечённой незнакомой местности с визуальным распознаванием цели навигации». Проект выполнен на средства выделенные «Фондом содействия развитию малых форм предприятий в научно-технической сфере» (Фонд содействия инновациям) по договору предоставления гранта № 64ГУКодИИС13-D7/102402 от 23 декабря 2024г.

Под полностью автономным режимом навигации (движения) в данном проекте понимается режим, при котором ровер с Аккермановой геометрией поворота самостоятельно, без команд оператора (человека), передвигается по умеренно пересечённой и незнакомой местности по указателям направления движения до указателя конечной цели, может выполнить заранее запрограммированные действия у каждого указателя и самостоятельно вернуться обратно к месту старта. При этом оператор может просматривать на своем мониторе видеоизображения и телеметрию, передаваемые с ровера. «Открытая библиотека автономной навигации по распознанным указателям движения» включает в себя 2D локализацию, картирование, построение маршрута с возможностью работы без использования глобальных систем спутникового позиционирования и управление движением ровера с помощью алгоритмов SLAM (Simultaneous Localization and Mapping) в реальных условиях по умеренно пересеченной местности.

«Открытая библиотека автономной навигации по распознанным указателям движения» разработана на языке программирования C++, для платформы ROS2 Humble (Robot Operating System 2 версии Humble).

СОДЕРЖАНИЕ.

АННОТАЦИЯ.....	2
СОДЕРЖАНИЕ.	3
Общие сведения о программе.	4
Структура программы.....	5
1. КЛАСС Localization.	5
2. КЛАСС Calculate_localization.	6
3. ФУНКЦИЯ get_location().	6
4. КЛАСС Mapping.....	7
5. КЛАСС Construction_map.....	7
6. ФУНКЦИЯ get_map().....	8
7. КЛАСС Navigation.	8
8. ФУНКЦИЯ move_to().	8
9. ФУНКЦИЯ setup().	9
Интеграция с модулем управления движением робота на низком уровне.	10
Интеграция с камерой глубины.	10
Настройка ROS2 Humble для работы библиотеки.	11
Настройка параметров навигации и локализации.	14
1. Настройка Nav2.....	14
2. Настройка колёсной одометрии.....	15
3. Настройка расширенного фильтра Калмана.....	15
4. Повторная сборка и проверка топиков.....	16
Примеры использования библиотеки.....	18
Пример программного кода для автономного движения робота по указателям движения (стрелкам) и по указателю конечной цели (конусу).....	18
Пошаговая инструкция запуска примера.....	18
Альтернативный пример программного кода для автономного движения робота до целевых точек с указанием целевой позиции.	21
Пошаговая инструкция запуска альтернативного примера.....	22

Общие сведения о программе.

«Открытая библиотека автономной навигации по распознанным указателям движения» предназначена для автоматического определения текущей позиции робота (2D локализация), для построения карты местности в режиме реального времени (картирование), для построения маршрута робота с учетом рельефа местности и возможностью работы без использования глобальных систем спутникового позиционирования (навигации), а также для управления движением робота с помощью алгоритмов SLAM (Simultaneous Localization and Mapping) в реальных условиях по умеренно пересеченной местности. Метод одновременной навигации, построения карты и движения увязывает независимые процессы в непрерывный цикл последовательных вычислений, при этом результаты одного процесса участвуют в вычислениях другого процесса. Это позволяет добиться полной автономности в движении робота по незнакомой местности.

МИНИМАЛЬНЫЕ ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ:

- Операционная система: Ubuntu 22.04.6,
- Процессор: AMD Ryzen R7 6800U,
- ОЗУ: 16Гб,
- Объем диска: 16 Гб,
- Наличие камеры глубины,
- Наличие IMU WITMOTION WT901BLECL BLE5.0,
- Наличие колесной базы с энкодерами на моторах.

ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ.

Платформа: ROS2 Humble (Robot Operating System 2 версии Humble).

Язык программирования: C++.

Зависимости:

- robot_localization,
- Nav2,
- Rtabmap,
- BehaviorTree.CPP,
- BehaviorTree.ROS2,
- realtime-tools,
- backward-ros,
- controller-interface,
- generate-parameter-library,
- ackermann-msgs,
- ROS2 (интеграция с роботом/ровером).

СПИСОК ОБЪЕКТОВ ДЛЯ ДОКУМЕНТИРОВАНИЯ.

1. Класс Localization - структура данных о положении робота.
2. Класс Calculate_localization - класс получения данных локализации.
3. Функция get_location() - получение данных о положении робота в пространстве.
4. Класс Mapping - структура данных с картой местности.
5. Класс Construction_map - класс построения карты местности.
6. Функция get_map - функция получения карты местности, построенной роботом при помощи стереокамеры.
7. Класс Navigation - класс выстраивания пайплайна действий для автономной езды робота до указанной точки.
8. Функция move_to() - функция, запускающая автономную навигацию робота до точки.
9. Функция setup() - функция для запуска навигации по стрелкам для робота.

Структура программы.

Модуль: eureka_nav_lib.hpp

Публичные объекты библиотеки:

1. Класс Localization (структура данных),
2. Класс Calculate_localization (класс получения данных локализации),
3. Функция get_location() (получение данных о положении робота в пространстве),
4. Класс Mapping (структура данных),
5. Класс Construction_map (класс построения карты местности),
6. Функция get_map (функция получения карты местности, построенной роботом при помощи стереокамеры),
7. Класс Navigation (класс выстраивания пайплайна действий для автономной езды робота до указанной точки),
8. Функция move_to() (функция для запуска автономной навигации до точки),
9. Функция setup() (функция для запуска автономной навигации по стрелкам/конусу для робота).

1. КЛАСС Localization.

Описание: структура данных для хранения данных о местоположении робота в пространстве.

Тип: dataclass.

Поля (атрибуты).

- sec: int32
Временная метка сообщения в секундах
- nanosec: int32
Временная метка сообщения в наносекундах
- position: mas[float64, float64, float64]
Позиция робота в пространстве в формате (x, y, z)

- orientation: mas[float64, float64, float64, float64]
Ориентация ровера в пространстве в формате кватерниона (x, y, z, w)
- position_v: mas[float64, float64, float64]
Линейная скорость ровера в формате (x, y, z)
- orientation_v: mas[float64, float64, float64]
Линейная скорость ровера в формате (x, y, z)

Назначение: передача информации с модуля локализации ровера, для получения данных о положении ровера в пространстве, а также скорости ровера в режиме реального времени.

2. КЛАСС `Calculate_localization`.

Описание: основной класс для получения данных о локализации ровера.

Публичные методы:

- `get_location()` -> List[Localization]
Назначение: запрос на получение всех данных с модуля локализации.
Входные параметры: нет.
Возвращаемое значение: список переменных, для получения данных положения, ориентации, линейных и угловых скоростей ровера с указанием промежутка времени.

3. ФУНКЦИЯ `get_location()`.

- Полное имя: `Calculate_localization.get_location()`
- Назначение: запрос на получение всех данных с модуля локализации.
- Входные данные: нет.
- Выходные данные:
 - список результатов локализации (List[Localization]),
 - каждый результат содержит:
 - временную метку в секундах и наносекундах;
 - позицию ровера в пространстве;
 - ориентацию ровера в пространстве;
 - линейную и угловую скорость ровера.
- Алгоритм:
 - получение данных с IMU и колесной одометрии;
 - фильтрация и фьюз данных с применение EKF (расширенного фильтра Калмана);
 - запрос данных из топики фильтрованной одометрии.
- Применение: функция для получения финальной отфильтрованной одометрии.

4. КЛАСС Mapping.

Описание: структура данных для хранения карты построенной ровером с помощью стереокамеры.

Тип: dataclass.

Поля (атрибуты).

- sec: int32
Временная метка сообщения в секундах
- nanosec: int32
Временная метка сообщения в наносекундах
- resolution: float32
Размер ячейки в метрах
- width: uint32
Размер ширины карты в ячейках
- height: uint32
Размер высоты карты в ячейках
- position: mas[float64, float64, float64]
Позиция левого нижнего угла карты в пространстве в формате (x, y, z)
- orientation: mas[float64, float64, float64, float64]
Ориентация карты в пространстве в формате кватерниона (x, y, z, w)
- data: mas[int8...]
Значения ячеек карты в диапазоне (-1 - 100), где:
 - 0 - свободное пространство;
 - 100 – препятствие;
 - -1 - неизвестное пространство;
 - 1-99 - близость к препятствию.

Назначение: хранение карты построенной ровером с препятствиями.

5. КЛАСС Construction_map.

Описание: основной класс для получения карты с препятствиями, построенной ровером.

Публичные методы:

- get_map() -> List[Mapping]
Назначение: Запрос на получение всех данных с модуля SLAM.
Входные параметры: нет.
Возвращаемое значение: список переменных, для получения карты с препятствиями в пространстве с указанием промежутка времени.

6. ФУНКЦИЯ `get_map()`.

- Полное имя: `Construction_map.get_map()`
- Назначение: запрос на получение всех данных с модуля SLAM.
- Входные данные: нет.
- Выходные данные:
 - список результатов построения карты препятствий (`List[Mapping]`);
 - каждый результат содержит:
 - временную метку в секундах и наносекундах,
 - размер ячейки карты,
 - размер карты по высоте и ширине в ячейках,
 - позицию карты в пространстве,
 - значения каждой ячейки карты, дающую информацию о препятствиях.
- Алгоритм:
 - получение данных о положении робота в пространстве;
 - получение изображения и данных глубины изображения со стереокамеры;
 - применение алгоритма RtabMap, для построения 3д карты пространства;
 - перевод карты в 2D в виде карты стоимости.
- Применение: функция для получения финальной карты, позволяющая получать информацию о ландшафте местности.

7. КЛАСС `Navigation`.

Описание: основной класс запуска навигации робота, с возможностью задачи точки или запуска движения по стрелкам.

Публичные методы:

- `move_to(coordinates: x, y, z)`
Назначение: запуск навигации для достижения роботом, указанной точки.
Входные параметры: `coordinates` (координаты целевой точки в пространстве `x`, `y` и углу по `z`).
- `setup()`
Запуск навигации по стрелкам с применением детекции стрелок и алгоритмов навигации.

8. ФУНКЦИЯ `move_to()`.

- Полное имя: `Navigation.move_to()`
- Назначение: функция запуска навигации до целевой точки с указанием целевой позиции.
- Входные данные: `coordinates` (координаты целевой точки в формате координат `x` и `y` и угла по `z`).

- Входные данные: нет.
- Алгоритм:
 - получение данных локализации;
 - получение данных о целевой позиции;
 - построение глобальной траектории до целевой точки;
 - построение карты местности;
 - генерация вектора скоростей, состоящих из двух линейных по осям X и Y и одной угловой по оси Z , контроллером для управления ровером.
- Применение: функция запускает автономную навигацию до целевой точки с указанием целевой позиции.

9. ФУНКЦИЯ `setup()`.

- Полное имя: `Navigation.setup()`
- Назначение: функция запуска навигации по стрелкам с применением дерева поведения и генерацией целевых точек.
- Входные данные: нет.
- Алгоритм:
 - запуск дерева поведения передачей переменной инициализации в топик `/init`;
 - запуск алгоритма распознавания стрелок и конуса;
 - детекция стрелки на изображении;
 - расчет параметров для расчета целевой точки ровера;
 - получение данных локализации;
 - получение данных о целевой позиции;
 - построение глобальной траектории до целевой точки;
 - построение карты местности;
 - остановка движения ровера вблизи стрелки на 10 сек.003В
 - детекция конуса и остановка движения ровера вблизи конуса.
 - генерация вектора скоростей, состоящих из двух линейных по осям X и Y и одной угловой по оси Z , контроллером для управления ровером.
- Применение: функция запускает автономную навигацию по указателям движения (стрелкам) и по указателю конечной цели (конусу).

Интеграция с модулем управления движением ровера на низком уровне.

«Библиотека автономной навигации по распознанным указателям движения» интегрирована с модулем движения ровера на низком уровне («Библиотека управления движения ровером на низком уровне»). При движении ровера, в режиме реального времени, две данные библиотеки обмениваются следующей информацией.

- Входные данные, получаемые из модуля движения ровера на низком уровне:
 - `wheel_states` - угловые скорости колёс для расчета колесной одометрии.
- Выходные данные, передаваемые в модуль движения ровера на низком уровне:
 - `cmd_vel` - линейные скорости по осям X и Y и угловая скорость ровера по оси Z в формате `[vel_x, vel_y, ang_z]`.

Интеграция с камерой глубины.

«Библиотека автономной навигации по распознанным указателям движения» интегрирована с камерой глубины, установленной на ровере и подключенной к его главному компьютеру. При движении ровера, в режиме реального времени данные с камеры глубины поступают в «Библиотеку автономной навигации по распознанным указателям движения».

- Входные данные, получаемые с камеры глубины:
 - топик `/camera/camera/depth/image_rect_raw` — выравненное и калиброванное изображение глубины (карта расстояний),
 - - топик `/camera/camera/aligned_depth_to_color/image_raw` — глубина, наложенная на RGB-кадр,
 - - топик `/camera/camera/depth/color/points` — 3D-облако точек, которое используется блоками навигации (Nav2) для обхода препятствий.

В нашем ровере мы использовали камеру глубины Intel RealSense d435i. Чтобы передавать данные с этой камеры глубины только в объеме, указанном выше, мы запускаем ее официальный драйвер ('`realsense2_camera`') через дополнительно разработанный нами пакет '[eureka_camera_2](#)'. Данный пакет не является частью проекта «Разработки открытых библиотек для автономной навигации шестиколесного прототипа марсохода (ровера) по умеренно пересечённой незнакомой местности с визуальным распознаванием цели навигации», а процесс его установки приведен в данной инструкции для примера. Поэтому все команды установки камеры глубины RealSense в данной инструкции имеют пометку в комментариях: **'# ДЛЯ ПРИМЕРА!!!'**.

Настройка ROS2 Humble для работы библиотеки.

ВНИМАНИЕ!!! Данная инструкция не заменяет документацию разработчиков/производителей ОС и оборудования. Поэтому перед установкой и настройкой ОС и оборудования внимательно изучите документацию и SDK разработчиков и производителей.

Установка зависимостей.

ROS2 Humble:

```
# Проверяем, что в системе используется UTF-8
locale

sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

# Проверяем, что в системе применилась UTF-8
locale

# Подключаем репозиторий universe, необходимый для пакетов
sudo apt install software-properties-common
sudo add-apt-repository universe

# Устанавливаем curl и добавляем официальный APT-источник ROS 2
sudo apt update && sudo apt install curl -y
export ROS_APT_SOURCE_VERSION=$(curl -s https://api.github.com/repos/ros-
infrastructure/ros-apt-source/releases/latest | grep -F "tag_name" | awk -F'"' '{print
$4}')
curl -L -o /tmp/ros2-apt-source.deb "https://github.com/ros-infrastructure/ros-apt-
source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-
source_${ROS_APT_SOURCE_VERSION}.${(. /etc/os-release && echo ${UBUNTU_CODENAME:-
${VERSION_CODENAME}})_all.deb"
sudo dpkg -i /tmp/ros2-apt-source.deb

# Обновляем список пакетов после добавления репозитория ROS 2
sudo apt update

# Обновляем установленные пакеты системы до актуальных версий
sudo apt upgrade

# Устанавливаем полную desktop-версию ROS 2 Humble
sudo apt install ros-humble-desktop

# Устанавливаем инструменты разработчика для ROS 2
sudo apt install ros-dev-tools
```

robot_localization:

```
# Устанавливаем пакет robot_localization для модуля локализации
sudo apt install ros-humble-robot-localization
```

Nav2:

```
# Устанавливаем Nav2 для модуля навигации
sudo apt install ros-humble-navigation2
sudo apt install ros-humble-nav2-bringup
```

Rtabmap:

```
# Устанавливаем RTAB-Map для модуля маппинга
sudo apt install ros-humble-rtabmap-ros
```

BehaviorTree.CPP:

```
# Устанавливаем библиотеку BehaviorTree.CPP v3
sudo apt install ros-humble-behaviortree-cpp-v3

# Устанавливаем дополнительные зависимости для сборки
sudo apt install ros-humble-realtime-tools
sudo apt install ros-humble-backward-ros
sudo apt install ros-humble-controller-interface
sudo apt install ros-humble-generate-parameter-library
sudo apt install ros-humble-ackermann-msgs
```

Устанавливаем официальный драйвер камеры глубины.

ВНИМАНИЕ!!! При установке официального драйвера камеры глубины необходимо строго следовать инструкции производителя устройства и разработчика SDK.

```
# ДЛЯ ПРИМЕРА!!! Устанавливаем официальный драйвер камеры RealSense для ROS 2
sudo apt install ros-humble-realsense2-camera

# Процесс установки официального драйвера камеры глубины Intel RealSense
# зависит от модели камеры и главного компьютера робота, поэтому выполнения одной лишь
# упомянутой выше команды может быть недостаточно - она может завершиться ошибками.
# При установке драйвера необходимо строго следовать официальной инструкции
# производителя и разработчика SDK.
```

Сборка библиотеки MSUROVERTEAM-SLAM.

создадим папку нашего рабочего пространства `ros2_ws` и папку `src` внутри папки рабочего пространства `ros2_ws`:

```
cd ~/
mkdir ros2_ws
cd ros2_ws
mkdir src
```

Переходим в папку `src` нашего рабочего пространства:

```
cd ~/ros2_ws/src
```

Клонируем репозиторий библиотеки в папку `src`:

```
git clone https://github.com/KodII-rover/MSUROVERTEAM-SLAM.git

# ДЛЯ ПРИМЕРА!!! Клонирование пакета управления камерой глубины eureka_camera_2
git clone https://github.com/andreyalex1/eureka_camera_2.git
```

Возвращаемся в корневой каталог рабочего пространства:

```
cd ~/ros2_ws
```

Перед сборкой необходимо выйти из всех активных виртуальных окружений Python. Использование `colcon build` внутри `venv`, `virtualenv` или `Conda` может привести к тому, что `ament_cmake` будет использовать Python из виртуального окружения, в котором отсутствует модуль `catkin_pkg`.

Если используется `venv` или `virtualenv`, выполните:

```
deactivate
```

Если используется `Conda`, выполните:

```
conda deactivate
```

Проверьте используемый интерпретатор:

```
which python3
```

При стандартной системной конфигурации команда должна вывести:

```
/usr/bin/python3
```

Проверить наличие модуля `catkin_pkg` можно командой:

```
python3 -c "import catkin_pkg; print(catkin_pkg.__file__)"
```

Если модуль отсутствует, установите его:

```
sudo apt install python3-catkin-pkg
```

После неудачной сборки необходимо удалить ранее созданные каталоги `build`, `install` и `log`, поскольку в них может быть сохранён путь к интерпретатору из виртуального окружения:

```
cd ~/ros2_ws  
rm -rf build/ install/ log/
```

Подключите системное окружение ROS 2 Humble и выполните сборку:

```
source /opt/ros/humble/setup.bash  
cd ~/ros2_ws  
colcon build  
  
# ДЛЯ ПРИМЕРА!!! Выполняем отдельную сборку пакета eureka_camera_2  
# для запуска камеры глубины RealSense  
colcon build --packages-select eureka_camera_2
```

После успешной сборки подключите собранное рабочее пространство:

```
source ~/ros2_ws/install/setup.bash
```

Предупреждения компилятора могут носить информационный характер. Однако при наличии сообщений `FAILED`, `CMake Error`, `ModuleNotFoundError` или ненулевого кода завершения сборка не считается успешно выполненной.

```
cd ..
```

Обновляем переменные окружения:

```
source /opt/ros/humble/setup.bash  
cd ~/ros2_ws
```

Запускаем билд пакета нашей библиотеки:

```
colcon build
```

Во время сборки могут возникнуть предупреждения на терминале. Игнорируйте их.

Настройка параметров навигации и локализации.

Перед запуском библиотеки необходимо привести параметры навигации, колёсной одометрии и EKF в соответствие с геометрией робота, названиями ROS-топиков и установленными датчиками.

Редактировать необходимо исходные конфигурационные файлы, расположенные в каталоге репозитория:

```
~/ros2_ws/src/MSUROVERTEAM-SLAM/
```

1. Настройка Nav2

Файл:

```
eureka_navigation/config/nav2.yaml
```

Открыть файл можно командой:

```
nano ~/ros2_ws/src/MSUROVERTEAM-SLAM/eureka_navigation/config/nav2.yaml
```

Необходимо проверить следующие группы параметров:

- `global_frame` — глобальная система координат. Для данной конфигурации используется `map`;
- `robot_base_frame` — система координат основания робота. Используется `base_footprint`;
- `odom_topic` — топик отфильтрованной одометрии. Используется `/odometry/filtered`;
- `controller_frequency` — частота работы локального контроллера в герцах;
- `v_max` — максимальная линейная скорость робота в метрах в секунду;
- `xy_goal_tolerance` — допустимая ошибка достижения целевой позиции в метрах;
- `yaw_goal_tolerance` — допустимая ошибка ориентации в радианах;
- `footprint` — координаты контура робота относительно `base_footprint`, задаваемые в метрах;

- `width` и `height` — размеры локальной и глобальной карт стоимости в метрах;
- `resolution` — размер одной ячейки карты стоимости в метрах;
- `inflation_radius` — радиус расширения препятствий с учётом безопасного расстояния;
- `minimum_turning_radius` — минимальный физически достижимый радиус поворота ровера;
- `reverse_penalty`, `change_penalty` и `cost_penalty` — штрафы планировщика за движение задним ходом, изменение направления и приближение к препятствиям.

Контур `footprint` должен соответствовать фактическим габаритам ровера. Значения максимальной скорости и минимального радиуса поворота должны быть определены экспериментально с учётом кинематики шасси.

2. Настройка колёсной одометрии

Файл параметров узла `eureka_odometry` в текущей структуре репозитория расположен по адресу:

```
eureka_localization/config/odometry.yaml
```

Открыть файл можно командой:

```
nano ~/ros2_ws/src/MSUROVERTEAM-SLAM/eureka_localization/config/odometry.yaml
```

Необходимо настроить следующие параметры:

- `joint_sub_topic` — топик с положениями рулевых приводов и скоростями колёс, по умолчанию `/wheel_states`;
- `imu_sub_topic` — топик IMU, по умолчанию `/imu/data`;
- `wheel_radius` — фактический радиус колеса в метрах;
- `wheel_base` — расстояние между передней и задней рулевыми осями в метрах;
- `wheel_track` — расстояние между центрами левых и правых колёс в метрах;
- `measure_error` — экспериментальный относительный коэффициент коррекции скорости. Значение `0.0` соответствует отсутствию коррекции;
- `enable_odom_tf` — разрешение публикации TF колёсной одометрией.

При использовании EKF параметр `enable_odom_tf` рекомендуется оставить равным `false`, поскольку трансформацию `odom` → `base_footprint` публикует узел `robot_localization`. Одновременная публикация одной TF-трансформации двумя узлами не допускается.

3. Настройка расширенного фильтра Калмана

Файл:

```
eureka_localization/config/ekf_el_classico.yaml
```

Открыть файл можно командой:

```
nano ~/ros2_ws/src/MSUROVERTEAM-SLAM/eureka_localization/config/ekf_el_classico.yaml
```

Необходимо проверить следующие параметры:

- `frequency` — частота расчёта EKF в герцах;
- `map_frame` — глобальная система координат `map`;
- `odom_frame` — локальная система координат `odom`;
- `base_link_frame` — система координат основания ровера `base_footprint`;
- `world_frame` — система координат, относительно которой EKF формирует результат; в текущей конфигурации используется `odom`;
- `odom0` — входной топик колёсной одометрии `/eureka_odometry/odometry`;
- `imu0` — входной топик IMU `/imu/data`;
- `publish_tf` — публикация трансформации `odom` → `base_footprint`;
- `two_d_mode` — включение двумерной модели движения. Значение `true` применяется для плоской поверхности, `false` — если необходимо учитывать крен и тангаж;
- `imu0_remove_gravitational_acceleration` — удаление гравитационной составляющей из линейного ускорения IMU.

Массивы `odom0_config` и `imu0_config` определяют используемые компоненты измерений в следующем порядке:

```
[x, y, z,  
 roll, pitch, yaw,  
 vx, vy, vz,  
 vroll, vpitch, vyaw,  
 ax, ay, az]
```

Значение `true` включает соответствующую компоненту в EKF, значение `false` исключает её.

4. Повторная сборка и проверка топиков

Перед запуском необходимо проверить наличие входных топиков:

```
ros2 topic list | grep -E "wheel_states|imu "
```

После изменения конфигурационных файлов необходимо повторно собрать соответствующие пакеты:

```
cd ~/ros2_ws  
source /opt/ros/humble/setup.bash  
  
# ДЛЯ ПРИМЕРА!!! Повторно собираем пакет работы с камерой глубины RealSense  
colcon build --packages-select eureka_camera_2  
  
# Повторно собираем остальные пакеты  
colcon build --packages-select eureka_navigation eureka_localization eureka_odometry  
source install/setup.bash
```

После запуска навигационного стека рекомендуется проверить работу локализации:

```
ros2 topic hz /odometry/filtered
ros2 run tf2_ros tf2_echo odom base_footprint
ros2 topic hz /map
ros2 param dump /ekf_filter_node
ros2 param dump /controller_server
```

Наличие сообщений в /odometry/filtered, карты /map и непрерывной трансформации odom → base_footprint свидетельствует о корректном запуске основных компонентов локализации и картирования.

Перед запуском автономной навигации по распознанным указателям движения необходимо выполнить глобальный launch-файл, используя команду:

```
ros2 launch eureka_navigation nav2tune.launch.py
```

ВНИМАНИЕ!!! Если к главному компьютеру робота не подключено периферийное оборудование, описанное в разделе «Минимальные технические требования», или подключено оборудование, отличное от использованного при разработке проекта, то в процессе запуска библиотеки автономной навигации по распознанным указателям движения возникнут ошибки.

Также необходимо убедиться, что официальные ROS2-драйверы аппаратного обеспечения (включая камеру глубины) установлены корректно, запускаются и публикуют данные в соответствующие ROS2-топики независимо от данной библиотеки. Если эти требования не выполнены, то при запуске библиотеки автономной навигации по распознанным указателям движения также возникнут ошибки.

Примеры использования библиотеки.

Пример программного кода для автономного движения робота по указателям движения (стрелкам) и по указателю конечной цели (конусу).

С++ код:

```
#include "eureka_nav_lib/eureka_nav_lib.hpp"
#include <rclcpp/rclcpp.hpp>

class SetupNode : public rclcpp::Node {
public:
    SetupNode() : Node("setup_node") {
        // Инициализируем общую навигационную систему
        nav = std::make_shared<eureka::Navigation>(shared_from_this());

        // Запускаем функцию движения по стрелкам с остановкой в 10 секунд
        // у каждой стрелки и завершением движения возле конуса.
        nav->setup();

        // В данном примере просто выводим на консоль сообщение о завершении миссии.
        RCLCPP_INFO(this->get_logger(), "Navigation system setup completed");
    }

private:
    std::shared_ptr<eureka::Navigation> nav;
};

int main(int argc, char** argv) {
    rclcpp::init(argc, argv);
    auto node = std::make_shared<SetupNode>();
    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}
```

Пошаговая инструкция запуска примера.

1. Открыть сессию в терминале и зайти в папку ros2_ws (рабочее пространство)
2. Запустите launch-файл общего стека навигации введя команду:

```
ros2 launch eureka_navigation nav2tune.launch.py
```

3. Откройте новую сессию терминала и запустите launch-файл дерева поведения введя команду:

```
ros2 launch eureka_bt strategy.launch.py
```

4. Для запуска примера вам также требуется создать пакет, для этого введите команду:

```
ros2 pkg create --build-type ament_cmake example
```

5. Найдите в пакете файл CMakeLists.txt и введите следующее:

```
cmake_minimum_required(VERSION 3.16)
project(example LANGUAGES CXX)
if(CMAKE_CXX_COMPILER_ID MATCHES "(GNU|Clang)")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
set(THIS_PACKAGE_INCLUDE_DEPENDS
  eureka_nav_lib
  rclcpp
  rcpputils
)

foreach(Dependency IN ITEMS ${THIS_PACKAGE_INCLUDE_DEPENDS})
  find_package(${Dependency} REQUIRED)
endforeach()

include_directories(include/)
add_executable(${PROJECT_NAME}
  src/example.cpp
)

ament_target_dependencies(${PROJECT_NAME}
  eureka_nav_lib
  rclcpp
  rcpputils
)

# INSTALL
install(TARGETS
  ${PROJECT_NAME}
  DESTINATION lib/${PROJECT_NAME})

ament_package()
```

6. Зайдите в папку src пакета для запуска примера и создайте файл example.cpp введя команду:

```
touch example.cpp
```

7. Вставьте код ниже в файл `example.cpp`:

```
#include "eureka_nav_lib/eureka_nav_lib.hpp"
#include <rclcpp/rclcpp.hpp>

class SetupNode : public rclcpp::Node {
public:
    SetupNode() : Node("setup_node") {
        // Оставляем конструктор пустым или для простых инициализаций
    }

    void init() {
        // Инициализируем навигацию здесь, когда shared_ptr уже создан
        nav = std::make_shared<eureka::Navigation>(shared_from_this());

        // Запуск настройки движения
        nav->setup();

        RCLCPP_INFO(this->get_logger(), "Navigation system setup completed");
    }

private:
    std::shared_ptr<eureka::Navigation> nav;
};

int main(int argc, char** argv) {
    rclcpp::init(argc, argv);

    auto node = std::make_shared<SetupNode>();
    node->init(); // Безопасный вызов после создания shared_ptr

    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}
```

8. Вернитесь в директорию `ros2_ws` и соберите пакет введя команду:

```
colcon build --packages-select example
```

9. Далее введите команду `source install/setup.bash` для обновления файлов вашего рабочего пространства

10. Запустите пакет с примером введя команду:

```
ros2 run example example
```

Альтернативный пример программного кода для автономного движения робота до целевых точек с указанием целевой позиции.

C++ код:

```
#include "eureka_nav_lib/eureka_nav_lib.hpp"
#include <rclcpp/rclcpp.hpp>
#include <chrono>
#include <memory>

class SimpleNavigation : public rclcpp::Node {
public:
    SimpleNavigation() : Node("simple_navigation") {
        // We will initialize components in a separate method to ensure
        // the shared_ptr to 'this' is fully constructed.
    }

    void init() {
        // Initialize navigation library classes
        nav = std::make_shared<eureka::Navigation>(shared_from_this());
        loc = std::make_shared<eureka::Calculate_localization>(shared_from_this());
        map = std::make_shared<eureka::Construction_map>(shared_from_this());

        // Define the sequence of goal points
        // 45 seconds is an example interval between goals
        timer_ = this->create_wall_timer(
            std::chrono::seconds(45),
            [this]() {
                static int goal_num = 0;
                send_goal(goal_num);
                goal_num = (goal_num + 1) % 4;
            }
        );

        RCLCPP_INFO(this->get_logger(), "Navigation sequence started");
    }

private:
    std::shared_ptr<eureka::Navigation> nav;
    std::shared_ptr<eureka::Calculate_localization> loc;
    std::shared_ptr<eureka::Construction_map> map;
    rclcpp::TimerBase::SharedPtr timer_;

    void send_goal(int goal_id) {
        // Target positions: {x, y, yaw}
        double goals[4][3] = {
            {1.0, 0.0, 0.0},
            {2.0, 1.0, 1.57},
            {1.0, 2.0, 3.14},
            {0.0, 1.0, -1.57}
        };

        RCLCPP_INFO(this->get_logger(), "Sending goal %d: x=%.2f, y=%.2f",
            goal_id, goals[goal_id][0], goals[goal_id][1]);

        // Trigger navigation to the target position
        nav->move_to(goals[goal_id][0], goals[goal_id][1], goals[goal_id][2]);
    }
};
```

```
int main(int argc, char** argv) {
    rclcpp::init(argc, argv);

    auto node = std::make_shared<SimpleNavigation>();
    node->init(); // Safe call after shared_ptr creation

    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}
```

Пошаговая инструкция запуска альтернативного примера.

1. Открыть сессию в терминале и зайти в папку ros2_ws (рабочее пространство)
2. Запустите launch-файл общего стека навигации введя команду:

```
ros2 launch eureka_navigation nav2tune.launch.py
```

3. Для запуска примера вам также требуется создать пакет, для этого введите команду:

```
ros2 pkg create --build-type ament_cmake example
```

4. Найдите в пакете файл CMakeLists.txt и введите следующее:

```
cmake_minimum_required(VERSION 3.16)
project(example LANGUAGES CXX)
if(CMAKE_CXX_COMPILER_ID MATCHES "(GNU|Clang)")
    add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
set(THIS_PACKAGE_INCLUDE_DEPENDS
    eureka_nav_lib
    rclcpp
    rcpputils
)

foreach(Dependency IN ITEMS ${THIS_PACKAGE_INCLUDE_DEPENDS})
    find_package(${Dependency} REQUIRED)
endforeach()

include_directories(include/)
add_executable(${PROJECT_NAME}
    src/example.cpp
)

ament_target_dependencies(${PROJECT_NAME}
    eureka_nav_lib
    rclcpp
    rcpputils
)

# INSTALL
install(TARGETS
    ${PROJECT_NAME}
    DESTINATION lib/${PROJECT_NAME})

ament_package()
```

5. Зайдите в папку src пакета для запуска примера и создайте файл example.cpp введя команду:

```
t touch example.cpp
```

6. Вставьте код ниже в файл example.cpp:

```
#include "eureka_nav_lib/eureka_nav_lib.hpp"
#include <rclcpp/rclcpp.hpp>
#include <chrono>
#include <memory>

// Пример реализации автономной навигации до целевой точки.
class SimpleNavigation : public rclcpp::Node {
public:
    SimpleNavigation() : Node("simple_navigation") {
        // Конструктор остается пустым для корректной работы shared_from_this()
    }

    void init() {
        // Инициализируем компоненты библиотеки навигации
        nav = std::make_shared<eureka::Navigation>(shared_from_this());
        loc = std::make_shared<eureka::Calculate_localization>(shared_from_this());
        map = std::make_shared<eureka::Construction_map>(shared_from_this());

        // Определяем последовательность прохода ровером целевых точек
        timer_ = this->create_wall_timer(
            std::chrono::seconds(45),
            [this]() {
                static int goal_num = 0;
                send_goal(goal_num);
                goal_num = (goal_num + 1) % 4;
            }
        );

        RCLCPP_INFO(this->get_logger(), "Система навигации инициализирована.");
    }

private:
    std::shared_ptr<eureka::Navigation> nav;
    std::shared_ptr<eureka::Calculate_localization> loc;
    std::shared_ptr<eureka::Construction_map> map;
    rclcpp::TimerBase::SharedPtr timer_;

    void send_goal(int goal_id) {
        double goals[4][3] = {
            {1.0, 0.0, 0.0},
            {2.0, 1.0, 1.57},
            {1.0, 2.0, 3.14},
            {0.0, 1.0, -1.57}
        };

        RCLCPP_INFO(this->get_logger(), "Едем к точке %d: x=%.2f, y=%.2f",
            goal_id, goals[goal_id][0], goals[goal_id][1]);

        // Вызываем функцию движения
        nav->move_to(goals[goal_id][0], goals[goal_id][1], goals[goal_id][2]);
    }
};
```

```
int main(int argc, char** argv) {
    rclcpp::init(argc, argv);

    // Создаем узел через shared_ptr
    auto node = std::make_shared<SimpleNavigation>();

    // Вызываем инициализацию ПОСЛЕ создания shared_ptr
    node->init();

    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}
```

7. Вернитесь в директорию `ros2_ws` и соберите пакет введя команду:

```
colcon build --packages-select example
```

8. Далее введите команду `source install/setup.bash` для обновления файлов вашего рабочего пространства

9. Запустите пакет с примером введя команду:

```
ros2 run example example
```